

Innovatech Solutions

Plataforma Inteligente para la gestión integral de proyectos tecnológicos

InnoVaTech

Integrantes:

Diego Herrera
Luis Rosales

Contenido

Contexto	3
Patrón de arquitectura	3
Requerimientos funcionales del proyecto	4
1. Gestión de Proyectos	4
2. Gestión de Recursos y Colaboración	4
3. Monitoreo y Analítica	4
Requerimientos no funcionales del proyecto	5
1. Seguridad	5
2. Escalabilidad	5
3. Disponibilidad	5
4. Rendimiento	5
5. Mantenibilidad	5
6. Observabilidad	5
Casos de Uso del Sistema	6
Caso de Uso 1: Crear Proyecto	6
Caso de Uso 2: Asignar Recursos	6
Caso de Uso 3: Consultar KPI	6
Diagrama de contenedores	7
1. Gestión de Proyectos	7
2. Gestión de Recursos y Colaboración	7
3. Monitoreo y Analítica	7
4. Observabilidad y Logs (lo que mencionaste de New Relic)	7
Herramientas del proyecto	8
Frontend	8
Backend y Microservicios	8
Bases de Datos	8
Monitoreo y Observabilidad	8
1. Análisis ético de la arquitectura actual	9
1.1 Seguridad	9
1.2 Privacidad y tratamiento de datos	9
1.3 Sostenibilidad y responsabilidad ambiental	10
2. Identificación de riesgos éticos	10
3. Ajustes y mejoras propuestas	10
1. Evaluación general del diseño	11
2. Privacidad de los datos de los usuarios	11
3. Seguridad del sistema	11
4. Sostenibilidad computacional	12
5. Conclusión ética	12

Contexto

Innovatech Solutions es una empresa que se dedica al desarrollo de software y consultoría tecnológica, que atiende a clientes de diversos sectores tales como retail, fintech y organizaciones públicas y privadas. Con más de 120 profesionales distribuidos en equipos multidisciplinarios, la compañía está experimentando un crecimiento acelerado en los últimos años. Este crecimiento ha generado nuevos desafíos: la coordinación simultánea de múltiples proyectos, la gestión de forma eficiente de recursos humanos que sean distribuidos geográficamente y también la necesidad de contar con indicadores de desempeño en tiempo real para la toma de decisiones estratégicas

Ante estas dificultades, la organización ha decidido trabajar en la implementación de una plataforma tecnológica integrada que esté basada en microservicios, que permita a los usuarios centralizar la gestión de proyectos, mejorar la colaboración entre equipos y ofrecer herramientas analíticas para el monitoreo del desempeño. La solución busca garantizar escalabilidad, flexibilidad y resiliencia, alineándose con las mejores prácticas de arquitectura moderna.

El diseño preliminar de la plataforma se representa en un diagrama de componentes, donde se visualizan las principales piezas del sistema y sus interacciones. Dicho diagrama muestra cómo los usuarios acceden a esta plataforma mediante aplicaciones web y móviles, la gestión de proyectos, un KPI Service para el cálculo de indicadores y un Report Service para la generación de reportes y dashboards. Estos servicios se apoyan en bases de datos independientes, optimizadas para el almacenamiento transaccional y analítico, respectivamente.

De esta manera, el contexto del proyecto y la propuesta arquitectónica se integran en una visión clara: Innovatech Solutions busca transformar su gestión interna mediante una plataforma modular y escalable, capaz de responder a las exigencias de un entorno tecnológico altamente dinámico.

Patrón de arquitectura

El patrón de arquitectura adecuado para este proyecto es la arquitectura de Microservicios.

Pensando en la alta demanda de la que dispondrá, una estructura desacoplada permitirá una respuesta segura en cualquier momento. Además, esta estructura permite que se pueda escalar de forma futura en base a las necesidades que pueda tener la empresa de Innovatech al incorporar nuevas funcionalidades o servicios. Si bien la arquitectura principal es de microservicios, también es necesaria una arquitectura event-driven para el manejo de ciertos microservicios. Pensando en una estructura sólida que incluya capas como circuit breaker y message broker para intercambio asíncrono y también síncrono, la mejor combinación es una que complementa la modularidad de los microservicios con la operatividad de los servicios basados en eventos. También se utilizan patrones complementarios como patrones de observabilidad y patrón IAM para logeos.

Requerimientos funcionales del proyecto

1. Gestión de Proyectos

- Crear, editar y eliminar proyectos.
- Definir tareas dentro de cada proyecto.
- Asignar responsables y establecer fechas de inicio y término.
- Controlar estados de avance (pendiente, en progreso, completado).
- Registrar comentarios y adjuntar documentación asociada a cada tarea.
- Visualizar el progreso general del proyecto en tiempo real.

2. Gestión de Recursos y Colaboración

- Registrar y administrar perfiles de profesionales (rol, habilidades, disponibilidad).
- Asignar recursos humanos a proyectos según disponibilidad y competencias.
- Gestionar equipos multidisciplinarios distribuidos geográficamente.
- Proporcionar un espacio de comunicación interna (notificaciones, mensajes).
- Integrar acceso desde aplicaciones web y móviles para facilitar la colaboración.
- Mostrar la disponibilidad de cada recurso en un calendario compartido.

3. Monitoreo y Analítica

- Calcular indicadores clave de desempeño (KPI), tales como:
- Avance de proyectos (% completado).
- Utilización de recursos (horas asignadas vs. disponibles).
- Productividad por equipo o área.
- Generar reportes en formatos PDF/Excel.
- Proporcionar dashboards interactivos para directivos.
- Enviar notificaciones automáticas sobre hitos, retrasos o alertas de desempeño.
- Permitir la exportación de métricas para análisis externo.
- Integrar un sistema de observabilidad para monitorear el rendimiento de los microservicios.
- Registrar logs centralizados de eventos y errores para facilitar auditoría y depuración.
- Detectar y dar continuidad a errores recurrentes, permitiendo alertas proactivas y trazabilidad de fallos.

Requerimientos no funcionales del proyecto

1. Seguridad

- Autenticación y autorización de usuarios mediante roles (gestores, profesionales, directivos).
- Cifrado de datos en tránsito (HTTPS/SSL) y en reposo (PostgreSQL con encriptación).
- Registro de auditoría para todas las operaciones críticas (creación, edición, eliminación de proyectos).
- Protección contra ataques comunes (SQL Injection, XSS, CSRF).

2. Escalabilidad

- La arquitectura basada en microservicios permite escalar cada módulo de manera independiente.
- Uso de contenedores (Docker) para desplegar servicios en diferentes entornos.
- Posibilidad de integrar orquestadores como Kubernetes para escalar automáticamente según la carga.

3. Disponibilidad

- Los servicios deben estar disponibles al menos un 99% del tiempo en entornos de prueba.
- Implementación de Circuit Breaker para manejar fallos en la comunicación entre microservicios.
- Uso de réplicas en bases de datos para asegurar continuidad en caso de fallos.

4. Rendimiento

- Respuesta de las APIs en menos de 2 segundos en operaciones comunes (crear proyecto, asignar tarea).
- Dashboards y reportes deben generarse en menos de 5 segundos para garantizar fluidez en la toma de decisiones.
- Optimización de consultas SQL mediante índices y normalización adecuada.

5. Mantenibilidad

- Código organizado siguiendo principios SOLID y buenas prácticas de desarrollo.
- Documentación clara de APIs y servicios (OpenAPI/Swagger).
- Uso de control de versiones (Git) para trazabilidad de cambios.

6. Observabilidad

- Integración con sistemas de monitoreo (New Relic, Prometheus, Grafana).
- Logs centralizados para detectar errores y dar continuidad a fallos recurrentes.
- Alertas proactivas en caso de anomalías de rendimiento o disponibilidad.

Casos de Uso del Sistema

Caso de Uso 1: Crear Proyecto

- Actor principal: Gestor de proyectos.
- Objetivo: Registrar un nuevo proyecto en la plataforma.
- Flujo principal:
 1. El gestor accede a la aplicación web.
 2. Selecciona la opción "Crear proyecto".
 3. Ingresa nombre, descripción, fechas y responsables.
 4. El sistema valida los datos y guarda el proyecto en la BD de proyectos.
 5. El sistema confirma la creación y muestra el proyecto en la lista.
- Microservicio involucrado: API Project Manager + BD Proyectos.

Ctrl+click para ver
diagrama
CASODEUSO1

Caso de Uso 2: Asignar Recursos

- Actor principal: Gestor de proyectos.
- Objetivo: Asignar profesionales disponibles a un proyecto.
- Flujo principal:
 1. El gestor accede a la aplicación web/móvil.
 2. Selecciona un proyecto y abre la sección de recursos.
 3. El sistema muestra la disponibilidad de profesionales (calendario compartido).
 4. El gestor asigna un recurso a una tarea.
 5. El sistema actualiza la BD de proyectos y notifica al profesional.
- Microservicio involucrado: API Project Manager + Aplicación Web/Móvil.

Ctrl+click para ver
diagrama
CASODEUSO2

Caso de Uso 3: Consultar KPI

- Actor principal: Director de Innovatech Solutions.
- Objetivo: Revisar indicadores de desempeño de proyectos.
- Flujo principal:
 1. El director accede al dashboard en la aplicación web.
 2. El sistema solicita métricas al KPI Service.
 3. El KPI Service consulta la BD KPI y calcula indicadores (avance, productividad, utilización).
 4. El sistema muestra gráficos y tablas en el dashboard.
 5. El director puede exportar los datos a PDF/Excel.
- Microservicio involucrado: KPI Service + Report Service + BD KPI.

Ctrl+click para ver
diagrama
CASODEUSO3

Diagrama de contenedores

Ctrl+click para ver diagrama

CONTENEDORES

1. Gestión de Proyectos

- **Aplicación Web** y **Aplicación Móvil** permiten crear, editar y visualizar proyectos/tareas.
- **API Project Manager** contiene la lógica para asignar responsables, controlar estados de avance y registrar comentarios.
- **Base de Datos de Gestión de Proyectos** almacena proyectos, tareas, usuarios y documentación asociada.

2. Gestión de Recursos y Colaboración

- **Aplicación Web/Móvil** muestran disponibilidad de profesionales y permiten comunicación interna (notificaciones, mensajes).
- **API Project Manager** gestiona perfiles de profesionales y asigna recursos a proyectos según disponibilidad.
- **El flujo de datos JSON/HTTPS** asegura que la colaboración sea posible desde cualquier dispositivo.

3. Monitoreo y Analítica

- **KPI Service** calcula indicadores clave de desempeño (avance, utilización de recursos, productividad).
- **Report Service** genera reportes en PDF/Excel y dashboards interactivos.
- **Base de Datos KPI Service** almacena métricas y estadísticas sintetizadas.
- **SMTP** envía notificaciones automáticas sobre hitos y alertas de desempeño.

4. Observabilidad y Logs (lo que mencionaste de New Relic)

- Se puede **integrar en el KPI Service y Report Service** un sistema de observabilidad (ej. New Relic, Prometheus, Grafana).
- Esto cubriría los requerimientos adicionales de **logueo centralizado y continuidad de errores**, asegurando trazabilidad y alertas proactivas.

Herramientas del proyecto

Frontend

- React → Framework open-source gratuito para interfaces web modernas
- Flutter → Opción gratuita para aplicaciones móviles multiplataforma
- Vercel → Despliegue del proyecto en plataforma gratuita.

Backend y Microservicios

- Node.js / Spring Boot → Crear servicios backend y APIs para procesar información.
- API Gateway NGINX → Opciones gratuitas para puertas de entrada a todos los microservicios
- Docker → Opción gratuita para contenerizar aplicaciones para ejecutarse igual en cualquier entorno
- Kubernetes (opcional) → Orquestador de contenedores Docker gratuito en producción.

Bases de Datos

- PostgreSQL → Almacenamiento estructurado de la información del sistema.

Monitoreo y Observabilidad

- New Relic → Monitoreo del rendimiento de aplicaciones.
- Prometheus + Grafana → Recolección para métricas y dashboards de visualización de datos.
- Elasticsearch → Centralizar y analizar logs del sistema.

1. Análisis ético de la arquitectura actual

La arquitectura propuesta para Innovatech Solutions se basa en un **patrón de microservicios**, utilizando contenedores Docker, orquestación con Kubernetes, APIs REST y bases de datos independientes. Desde una perspectiva ética, este diseño presenta fortalezas relevantes en términos de escalabilidad, modularidad y eficiencia, pero también expone debilidades que pueden generar riesgos para la seguridad, la privacidad y la sostenibilidad del sistema.

1.1 Seguridad

- **Fortalezas:**
 - Autenticación y autorización basada en roles, alineada con el principio de mínimo privilegio.
 - Cifrado de datos en tránsito y en reposo, reduciendo riesgos de exposición.
 - Uso de logs de auditoría para operaciones críticas, fortaleciendo la trazabilidad y la responsabilidad.
- **Debilidades éticas:**
 - Sistema de autenticación débil.
 - Ausencia de políticas de rotación de credenciales y autenticación multifactor (MFA).
 - Falta de un servicio de identidad centralizado (IAM), lo que puede generar accesos indebidos entre microservicios.

Estas debilidades representan un riesgo de negligencia técnica evitable, con posibles impactos en los derechos de usuarios y trabajadores.

1.2 Privacidad y tratamiento de datos

- **Datos sensibles tratados:**
 - Información personal de profesionales (roles, disponibilidad, productividad).
 - Métricas de desempeño con impacto potencial en evaluaciones laborales.
 - Logs y registros de actividad con identificadores personales.
- **Aspectos positivos:**
 - Segmentación de bases de datos.
 - Separación entre datos transaccionales y analíticos.
- **Brechas éticas:**
 - Falta de políticas de minimización de datos.
 - Ausencia de criterios de retención y eliminación de información.
 - No se especifica el uso ético de métricas personales.
 - No se menciona el cumplimiento de principios GDPR o normativas equivalentes.

El uso de métricas de productividad sin salvaguardas adecuadas puede derivar en vigilancia excesiva o uso punitivo de los datos.

1.3 Sostenibilidad y responsabilidad ambiental

- **Aspectos positivos:**
 - Escalabilidad controlada por microservicios.
 - Uso de tecnologías open source que evitan infraestructuras sobredimensionadas.
- **Aspectos débiles:**
 - No se evalúa el consumo energético del monitoreo y los contenedores.
 - Riesgo de escalado automático sin límites.
 - Posible sobrecarga por monitoreo intensivo sin políticas de optimización.

Desde una ética de la sostenibilidad, la eficiencia técnica debe ir acompañada de decisiones conscientes sobre el impacto ambiental.

2. Identificación de riesgos éticos

- **Riesgos de seguridad:**
 - Exposición de APIs sin rate limiting.
 - Falta de control de acceso entre microservicios.
 - Posibles fugas de datos a través de logs o reportes exportables.
- **Riesgos de privacidad y normativos:**
 - Uso de datos personales en KPIs sin anonimización.
 - Incumplimiento potencial de principios como minimización, finalidad y derecho al olvido.
 - Riesgo de sesgos en análisis de productividad.
- **Riesgos ambientales y sociales:**
 - Escalado sin límites definidos.
 - Uso continuo de dashboards sin criterios de eficiencia.
 - Falta de métricas de consumo de recursos.

3. Ajustes y mejoras propuestas

- **Seguridad:**
 - Incorporar un servicio IAM con OAuth2/OpenID Connect.
 - Implementar MFA para roles críticos.
 - Aplicar principios de Zero Trust y enmascaramiento de datos sensibles.
- **Privacidad:**
 - Anonimización o seudonimización de KPIs.
 - Definición de políticas de retención y eliminación de datos.
 - Transparencia hacia los usuarios sobre el uso de su información.
- **Sostenibilidad:**
 - Definir límites de escalado en Kubernetes.
 - Optimizar recolección de métricas y logs.
 - Incorporar dashboards de eficiencia operativa.

1. Evaluación general del diseño

El diseño arquitectónico propuesto para la plataforma de Innovatech Solutions se basa en una arquitectura de microservicios, lo que permite modularidad, escalabilidad y flexibilidad. Desde una perspectiva ética, el sistema presenta una base adecuada, pero requiere mejoras para garantizar una protección efectiva de los datos, un uso responsable de los recursos tecnológicos y una mayor alineación con principios éticos y normativos.

2. Privacidad de los datos de los usuarios

- El diseño considera medidas básicas de protección, como:
 - Autenticación y autorización por roles.
 - Cifrado de datos en tránsito y en reposo.
 - Separación de bases de datos por microservicio.
- Estas medidas ayudan a proteger la confidencialidad de la información.

Sin embargo:

- No se definen políticas claras de minimización de datos.
- No se especifica cuánto tiempo se almacenan los datos ni cuándo se eliminan.
- No se aclara el uso ético de métricas de desempeño individuales.

Desde una mirada ética, estas omisiones pueden afectar la privacidad de los profesionales, especialmente cuando se utilizan indicadores de productividad que podrían derivar en control excesivo o evaluaciones injustas.

3. Seguridad del sistema

- El sistema cumple parcialmente con principios éticos de seguridad al:
 - Utilizar cifrado de información.
 - Registrar acciones críticas mediante logs de auditoría.
- No obstante, se identifican debilidades importantes:
 - Falta de un sistema de identidad centralizado.
 - Ausencia de autenticación multifactor para roles críticos.
 - Falta de controles claros sobre el uso de las APIs.

Estas debilidades pueden facilitar accesos indebidos, abuso de servicios o interrupciones del sistema, lo que afecta tanto a los usuarios como a la organización.

4. Sostenibilidad computacional

- La arquitectura de microservicios permite:
 - Escalar solo los servicios necesarios.
 - Optimizar el uso de recursos computacionales.
- Aun así, el diseño no contempla:
 - Límites claros al escalado automático.
 - Métricas sobre consumo energético.
 - Políticas para optimizar el monitoreo y los logs.

Desde una ética de la sostenibilidad, el sistema debería considerar el impacto ambiental y el uso eficiente de la infraestructura tecnológica.

5. Conclusión ética

En conclusión, el diseño propuesto cumple de forma inicial con principios éticos básicos, especialmente en seguridad y estructura técnica. Sin embargo, la evaluación ética evidencia la necesidad de mejorar la protección de la privacidad, reforzar la seguridad y adoptar medidas de sostenibilidad. Incorporar estas mejoras permitirá un uso más responsable de la tecnología, alineado con la protección de las personas, la prevención del daño y la responsabilidad profesional.